

Observability

1. kubectl — Day-to-Day Usage (EKS / Kubernetes)

What it is

CLI to interact with Kubernetes clusters (including **Amazon EKS**).

kubectl + Datadog — Day-to-Day Developer Guide (EKS)

```
=====
SECTION 1: kubectl (Kubernetes CLI)
=====
```

Mental Model

Kubernetes resources you interact with:

- Pod → running container
- Deployment → manages pods (replicas, rollout)
- Service → networking
- ConfigMap → config
- Secret → sensitive config

```
-----
# 1. Setup (EKS)
-----
```

Configure kubectl for EKS cluster

```
aws eks update-kubeconfig --region <region> --name <cluster-name>
```

Verify connection

```
kubectl get nodes
```

```
-----
# 2. Basic Commands (Most Used Daily)
-----
```

List pods

```
kubectl get pods -n <namespace>
```

Wide output (IP, node, etc.)

```
kubectl get pods -o wide
```

List deployments

```
kubectl get deployments
```

List all resources
kubectl get all

3. Debugging Pods (CRITICAL)

Describe pod (events + errors)
kubectl describe pod <pod-name>

Common things to check:
- CrashLoopBackOff
- ImagePullBackOff
- OOMKilled
- Liveness probe failure

4. Logs (VERY FREQUENT)

View logs
kubectl logs <pod-name>

Follow logs (like tail -f)
kubectl logs -f <pod-name>

Multi-container pod
kubectl logs <pod-name> -c <container-name>

Previous crash logs
kubectl logs <pod-name> --previous

5. Exec into Pod (DEEP DEBUGGING)

kubectl exec -it <pod-name> -- /bin/sh

Inside container:
- check env vars
- test internal APIs
- inspect files

6. Restart / Fix Deployment

Restart deployment
kubectl rollout restart deployment <deployment-name>

Scale deployment
kubectl scale deployment <deployment-name> --replicas=3

Delete pod (auto recreated)
kubectl delete pod <pod-name>

7. Apply Changes

Apply config
kubectl apply -f deployment.yaml

Delete resource
kubectl delete -f deployment.yaml

8. Port Forwarding (Local Debugging)

kubectl port-forward pod/<pod-name> 8080:8080

Access locally:
http://localhost:8080

9. Common Failure Cases (EDGE CASES)

Pod stuck in Pending
→ Node capacity issue / scheduling issue

CrashLoopBackOff
→ App crashing repeatedly

OOMKilled
→ Memory limit exceeded

ImagePullBackOff
→ Wrong image / auth issue

No logs
→ Container not starting

10. Typical Debug Flow (kubectl)

```
kubectl get pods
kubectl describe pod <pod>
kubectl logs <pod>
kubectl exec -it <pod> -- /bin/sh
```

SECTION 2: Datadog (Observability)

Signals in Datadog

- Logs → What happened
- Metrics → System health
- Traces → Request flow

1. Logs (MOST USED)

Search queries:

```
service:payment status:error
service:user-api @http.status_code:500
trace_id:abc123
```

Use cases:

- Debug exceptions
- Find request failures
- Investigate production bugs

2. Metrics (SYSTEM HEALTH)

Common metrics:

- CPU usage
- Memory usage
- Request latency (p95, p99)
- Error rate

Example:

```
"Is latency increasing?"
"Is CPU spiking?"
```

3. APM (Distributed Tracing)

Request flow:

API → service A → service B → DB

Identify:

- Slow service
- Bottleneck
- External dependency latency

4. Alerts

Examples:

- error_rate > 5%
- latency > threshold
- pod restarts

Notification:

- Slack
- Email

5. Service Map

Shows:

- All services
- Dependencies
- Traffic flow

6. Typical Debug Flow (Datadog)

1. Alert triggered
2. Open logs → find error
3. Extract trace_id
4. Open trace → find slow service
5. Check metrics → CPU/memory

SECTION 3: Combined Workflow (REAL WORLD)

Scenario: Production Issue

Step 1:

Datadog Alert Triggered

Step 2:

Check logs in Datadog

→ Identify error

Step 3:

Check trace

→ Identify failing service

Step 4:

Check metrics

→ CPU spike? latency?

Step 5:

If deeper issue:

kubectl get pods

kubectl describe pod

kubectl logs

kubectl exec

Step 6:

Fix issue

→ restart / deploy

SECTION 4: Responsibility Split

Datadog

- Observability
- Logs
- Metrics
- Traces
- Alerts

kubectl

- Control
- Debug infra
- Restart pods
- Deploy changes

SECTION 5: What You Use Daily

Very Frequent

- Datadog logs

- Datadog dashboards
- kubectl logs
- kubectl describe

Moderate

- kubectl exec
- Datadog tracing

Less Frequent

- kubectl apply
- scaling deployments

=====

FINAL SUMMARY

=====

Datadog → Detect + Analyze issues

kubectl → Fix + Control system

Both are required for real-world Kubernetes workflows.

kubectl + Datadog — Common Interview Q&A (Developer Focus)

=====

SECTION 1: FUNDAMENTALS

=====

Q1. What is kubectl?

A:

kubectl is the CLI used to interact with Kubernetes clusters.

It allows you to deploy, inspect, debug, and manage resources like pods, deployments, and services.

Q2. What is Datadog?

A:

Datadog is an observability platform that provides:

- Logs
- Metrics
- Distributed tracing (APM)
- Alerts

It helps monitor and debug applications in production.

Q3. Difference between kubectl and Datadog?

A:

kubectl → Control plane (operate cluster)

Datadog → Observability (monitor/debug system)

kubectl = "fix"

Datadog = "detect + analyze"

Q4. Can Datadog replace kubectl?

A:

No.

Datadog cannot:

- Restart pods
- Exec into containers
- Apply deployments

kubectl is mandatory for cluster operations.

=====

SECTION 2: REAL-WORLD DEBUGGING

=====

Q5. How do you debug a production issue?

A:

1. Datadog alert triggers
2. Check logs in Datadog
3. Identify error / trace_id
4. Use APM to trace request flow
5. Check metrics (CPU, memory, latency)
6. If needed:
 - kubectl get pods
 - kubectl describe pod
 - kubectl logs
 - kubectl exec
7. Fix and redeploy

Q6. Pod is in CrashLoopBackOff. What do you do?

A:

kubectl describe pod <pod>

kubectl logs <pod> --previous

Check:

- Application crash

- Wrong config
 - Missing env variables
-

Q7. Logs not visible in Datadog. What next?

A:

Use `kubectl logs <pod>`

Possible issues:

- Datadog agent not running
 - Log pipeline misconfigured
 - Delay in ingestion
-

Q8. How do you check if service is slow?

A:

Use Datadog APM:

- Trace request
- Identify slow service
- Check latency breakdown

Then correlate with:

- CPU/memory metrics
-

Q9. How do you debug high latency?

A:

1. Datadog APM → find slow span
2. Check downstream dependencies
3. Verify DB/external calls
4. If infra issue:
 - `kubectl top pods`
 - `kubectl describe pod`

SECTION 3: kubectl DEEP QUESTIONS

Q10. Difference between get, describe, logs?

A:

`kubectl get` → list resources (high-level)

kubectl describe → detailed info + events
kubectl logs → application logs

Q11. How do you exec into a pod?

A:
kubectl exec -it <pod> -- /bin/sh

Used for:
- Debugging runtime issues
- Checking files/config

Q12. How do you restart a pod?

A:
kubectl delete pod <pod>

OR

kubectl rollout restart deployment <deployment>

Q13. What happens when you delete a pod?

A:
If managed by deployment:
→ Kubernetes recreates it automatically

Q14. How do you scale an application?

A:
kubectl scale deployment <name> --replicas=5

Q15. How do you apply changes?

A:
kubectl apply -f deployment.yaml

SECTION 4: DATADOG DEEP QUESTIONS

Q16. What are logs, metrics, and traces?

A:

Logs → discrete events (errors, messages)

Metrics → numeric time-series (CPU, latency)

Traces → request journey across services

Q17. What is APM?

A:

Application Performance Monitoring:

Tracks requests across microservices and shows latency breakdown.

Q18. How does Datadog collect data?

A:

- Agent installed on host/container
- APM libraries in application
- Integrations (Kubernetes, AWS)

Q19. What is a trace_id?

A:

Unique ID for a request across services.

Used to correlate logs, metrics, and traces.

Q20. How do you create alerts?

A:

Based on:

- Metrics (CPU, latency)
- Logs (error count)
- APM (error rate)

SECTION 5: SCENARIO-BASED QUESTIONS

Q21. Pod is running but API is failing. Why?

A:

Possible reasons:

- Application bug
- Wrong config
- Downstream service failure

Check:

Datadog logs → trace → kubectl logs

Q22. High CPU usage in pods. What do you do?

A:

1. Datadog metrics → confirm spike
 2. kubectl top pods
 3. Check logs
 4. Scale or optimize
-

Q23. How do you check environment variables in a pod?

A:

kubectl exec -it <pod> -- printenv

Q24. How do you debug networking issues?

A:

kubectl exec into pod
curl internal service

Also check:

- service config
 - DNS
-

Q25. When do you use kubectl over Datadog?

A:

- Need to modify cluster
- Debug container internals

- Restart or deploy services

=====

SECTION 6: TRICK / EDGE CASE QUESTIONS

=====

Q26. Why logs differ between Datadog and kubectl?

A:

- Datadog ingestion delay
 - Filtering rules
 - Misconfigured agent
-

Q27. Why pod has no logs?

A:

- Container crashes immediately
 - Logging not configured
 - Wrong container selected
-

Q28. Multiple containers in pod—how to get logs?

A:

kubectl logs <pod> -c <container>

Q29. Pod stuck in Pending?

A:

- No node capacity
 - Resource constraints
 - Taints/tolerations issue
-

Q30. OOMKilled means?

A:

Container exceeded memory limit and was killed.

=====

FINAL SUMMARY

=====

kubectl → execution + control
Datadog → visibility + analysis

Interview key:

Always explain flow:

"Detect in Datadog → Debug deeper with kubectl → Fix"

database scenario github

12 Factor of Microservice

1. Domain Decomposition (DDD)

- Use Domain-Driven Design
 - Identify **bounded contexts** and split services around business capabilities
 - Avoid technical-layer-based splitting (e.g., service per DAO)
-

2. Single Responsibility per Service

- Each service should own **one business capability**
 - Reduces coupling and simplifies scaling + maintenance
-

3. Loose Coupling & High Cohesion

- Services must be independent
 - Avoid shared databases
 - Communicate via APIs/events only
-

4. Database per Service

- Each microservice owns its data
 - No direct cross-service DB access
 - Ensures autonomy and independent deployments
-

5. Inter-Service Communication

- Sync: REST / gRPC
 - Async: messaging (Kafka, RabbitMQ)
 - Prefer async for scalability and fault tolerance
-

6. API Gateway

- Use a centralized entry point like Spring Cloud Gateway or NGINX
 - Handles routing, authentication, rate limiting
-

7. Service Discovery

- Dynamic service registration and lookup
 - Tools: Eureka, Consul
 - Required for scaling and dynamic environments
-

8. Fault Tolerance & Resilience

- Circuit breakers, retries, timeouts
 - Tools: Resilience4j
 - Prevent cascading failures
-

9. Observability (Monitoring + Logging + Tracing)

- Metrics: Prometheus
 - Visualization: Grafana
 - Tracing: Zipkin / Datadog
-

10. CI/CD Pipeline

- Automate build, test, deploy
 - Tools: Jenkins, GitLab
 - Enables frequent and safe deployments
-

11. Containerization & Orchestration

- Package services using Docker
 - Orchestrate with Kubernetes
 - Enables scaling, self-healing, portability
-

12. Security

- Authentication & authorization (OAuth2, JWT)
- Secure communication (HTTPS, mTLS)
- API Gateway often enforces security policies

AWS

Amazon EC2

- Hosts backend services, Kafka, Redis (if self-managed)

Amazon EKS (Kubernetes)

- Runs microservices as containers (pods)
- Handles scaling, deployment, orchestration

Amazon RDS (MySQL)

- Stores audit data, users, roles, findings

Amazon S3

- Stores files (audit evidence, reports, attachments)

Amazon ElastiCache (Redis)

- Caches roles, permissions, sessions for performance

Amazon MSK (Kafka)

- Handles async communication between services (event-driven)

AWS Lambda

- Runs background jobs, scheduled tasks, lightweight processing

Amazon API Gateway

- Entry point for frontend → backend APIs
- Handles routing, security, throttling

Amazon CloudWatch

- Logs, monitoring, alerts for services

Config

◆ Configuration & Secret Management in Our Application

- **Kubernetes ConfigMap**

- Used for **non-sensitive configuration**
 - Examples:
 - Service URLs (other microservices endpoints)
 - Kafka topics
 - Feature flags
 - Timeout/retry configs
 - Reason: Easy to update without code change, safe to expose
-

- **Kubernetes Secret**

- Used for **basic sensitive data inside cluster**
 - Examples:
 - Database username/password
 - JWT secret
 - Internal API keys
 - Reason: Native Kubernetes support, easy injection into pods
-

- **AWS Secrets Manager**

- Used for **highly sensitive and critical secrets (production)**
 - Examples:
 - Production DB credentials
 - Third-party API credentials
 - Encryption keys
 - Reason:
 - Strong security (KMS encryption)
 - Secret rotation
 - IAM-based access control
-

◆ Overall Approach

- ConfigMap → **non-sensitive configs**
- Kubernetes Secret → **basic secrets**
- AWS Secrets Manager → **secure, production-grade secrets**

Retry

```
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot2</artifactId>
</dependency>
```

```
<!-- Optional but recommended (for monitoring) -->
```

```
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-micrometer</artifactId>
</dependency>
```

```
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot2</artifactId>
</dependency>
```

```
<!-- Optional but recommended (for monitoring) -->
```

```
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-micrometer</artifactId>
</dependency>
```

Yml

resilience4j:

 retry:

 instances:

 notificationService: # Logical name (same as @Retry(name="..."))

 max-attempts: 3

 # Total attempts INCLUDING first call

 # Example: 3 = 1 original + 2 retries

 wait-duration: 2s

 # Delay between retries

 # Avoid 0 → can overload system

 enable-exponential-backoff: true

 # If true → delay increases each retry

 exponential-backoff-multiplier: 2

 # Delay pattern:

 # 1st retry → 2s

 # 2nd retry → 4s

 # 3rd retry → 8s

```
retry-exceptions:
  - java.io.IOException
  - java.net.SocketTimeoutException
# Only retry for these exceptions

ignore-exceptions:
  - java.lang.IllegalArgumentException
# NEVER retry these (business errors)

retry-exception-predicate: null
# Advanced: custom logic to decide retry

fail-after-max-attempts: true
# After all retries fail → throw exception
```

Example

=====

```
@Service
public class NotificationClient {

    private final RestTemplate restTemplate;

    public NotificationClient(RestTemplate restTemplate) {
        this.restTemplate = restTemplate;
    }

    @Retry(name = "notificationService", fallbackMethod = "sendFallback")
    public String sendNotification(String auditId) {

        System.out.println("Calling Notification Service...");

        // Simulating REST call
        String url = "http://notification-service/api/send/" + auditId;

        return restTemplate.getForObject(url, String.class);
    }

    // Fallback method (must have same params + Exception)
    public String sendFallback(String auditId, Exception ex) {
        System.out.println("Fallback triggered due to: " + ex.getMessage());
        return "Notification failed, fallback response";
    }
}
```